PROJECT A Teams of three

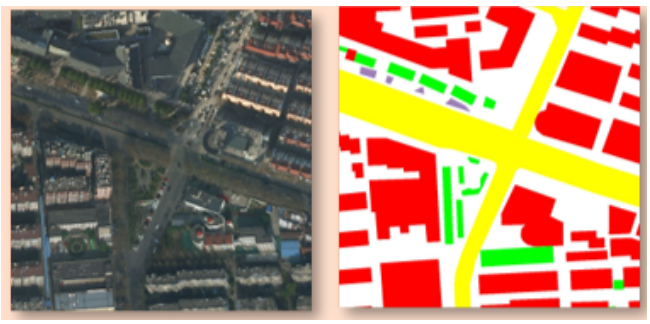
Think Again, Segment Better

Looped transformers for road segmentation

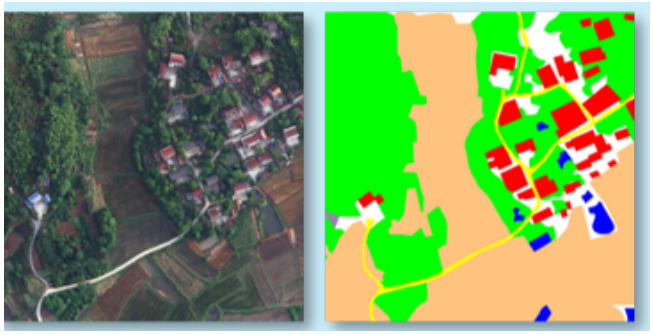
The research question. Can a small model improve a road segmentation by revisiting its current answer, using the *same learned update rule* several times? Your goal is to test when extra iterations repair mistakes, when they do not, and why.

Why roads?

A segmentation model assigns a label to every image pixel. For roads, a plausible-looking mask is not always a useful map: a short gap can break a route, a false bridge can connect unrelated streets, and a thick boundary can swallow a sidewalk. Local appearance alone can be ambiguous. Repeatedly combining image evidence with the current map may help resolve these ambiguities. That is a hypothesis to investigate, not a promised result.



Urban example: image and annotation



Rural example: image and annotation

Figure 1: High-resolution satellite images and land-cover annotations, cropped in the page layout from Fig. 1 of Wang et al. [1]. Roads are yellow; this project predicts road versus non-road only. Figure credit: Wang et al.; source paper CC BY-NC-SA 4.0. Obtain project data from the course-provided package.

What you will build

A **trainable recurrent transformer refiner** on top of a common, frozen initial segmenter. Submit one set of weights and code. Staff will run it for exactly $K = 1, 2, 4$ refinement steps for ranking, and $K = 0, 6, 8$ for diagnosis. The same checkpoint must support every K.

Release status. This handout is an instructor-review draft. Before launch, staff must publish the course data package, frozen baseline and feature interface, split manifests, final resource caps, and confirmed deadlines. The companion code includes an *untrained synthetic demo*, not a trained road segmenter. The proposed metrics and weights below are public; they must be frozen before competition starts.

Reading route: concepts and design, Sections 2–4; evaluation, Sections 5–8; submission, Section 9; research expectations and grading, Sections 10–11; readings and checklist, Section 12.

2 A short introduction to transformers and loops

Start with three ideas

Features are numerical descriptions learned from an image: edges, textures, or more abstract patterns. A **token** is a vector describing one location or patch. A **transformer block** lets tokens exchange information using attention, then updates each token with a small neural network. You do not need to have taken the transformer lectures to start this project.

For example, divide a feature map into N locations, each represented by a vector of length d. Put them in a matrix $X \in \mathbb{R}^{N \times d}$. One attention head computes

$$Q = XW_Q, \quad J = XW_J, \quad V = XW_V, \qquad \text{Attention}(X) = \text{softmax}_{\text{row}}\left(\frac{QJ^\top}{\sqrt{d_h}}\right)V. \tag{1}$$

Here W_Q, W_J, W_V are learned matrices; Q contains *queries*, J contains *keys*, V contains *values*, and d_h is the query/key vector length. Row-wise softmax converts each token's similarities into weights that sum to one. We use J for keys to avoid confusing them with the loop count K. Multiple heads learn different information exchanges. Position information tells the model where tokens are located.

Stacking is not the same as weight sharing

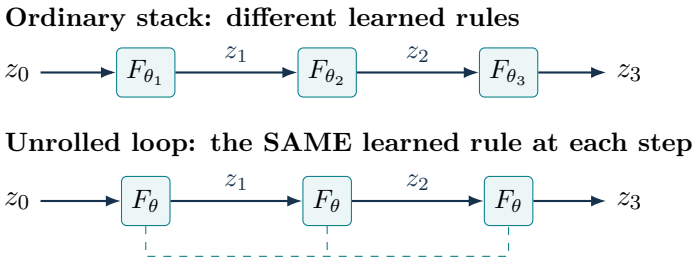


Figure 2: Unrolling draws repeated calls as separate boxes for readability. In the loop, every box refers to the same module and the same parameter tensors.

If one block has p parameters, K independent blocks have about Kp parameters. Reusing one block keeps its parameter count at p, although its computation still grows approximately with K. The hidden state z_t changes; the weights θ do not change during inference.

What counts as reasoning here? We use the narrower term *iterative spatial inference*: a model uses context and its current state to revise a prediction. Shared weights do not prove reasoning, and more loops do not guarantee a better answer. Your experiments must test whether state and feedback actually matter.

Universal Transformers motivate shared-depth attention [2]. Tiny Recursive Models motivate repeatedly updating a latent state and a candidate answer [3]. This project adapts these ideas; it is not a literal reproduction of either architecture.

3 The model contract: one image, a changing state

Two different things change at each iteration

For one image of height H and width W (omitting the batch dimension), t counts completed refinement steps:

- **The current answer, $y_t \in \mathbb{R}^{H\times W}$:** one real-valued *road logit* per pixel. A logit is an unrestricted score, not a probability: $p_t(u) = \sigma(y_t(u)) = 1/(1 + e^{-y_t(u)})$ is the probability of road at pixel u. Thus y_t is a dense prediction map, not a ground-truth label or a loss.
- **The latent state, $z_t \in \mathbb{R}^{N_z\times d}$:** a bank of N_z memory tokens, each a computed d-dimensional feature vector, not a road probability or model parameter. Tokens carry context between steps; a spatial state may keep one per coarse image patch. Their values change at inference; the model weights do not.

We reserve $g(u) \in \{0, 1\}$ for ground truth and L for loss. Ground truth never enters the inference loop. A frozen staff encoder and head produce $f = E(x)$ and $y_0 = H_0(f)$ from RGB image x. Your initializer I_ϕ and shared update R_θ give

$$z_0 = I_\phi(f, y_0), \tag{2}$$

$$(z_{t+1}, y_{t+1}) = R_\theta(f, z_t, y_t), \quad t = 0, \ldots, K-1, \tag{3}$$

$$p_K = \sigma(y_K), \qquad P_K(u) = \mathbf{1}[p_K(u) \geq 0.5]. \tag{4}$$

What the transformer sees. Inside R_θ, adapters T_x and T_y embed image features and the current answer as token matrices: $X = T_x(f) \in \mathbb{R}^{N_x\times d}$ and $T_y(y_t) \in \mathbb{R}^{N_y\times d}$. N_x, N_y count tokens; d is their vector length. y_t is *tokenized before attention* and the new answer is decoded back to $H \times W$ logits. The tokenizers and output head are shared across steps.

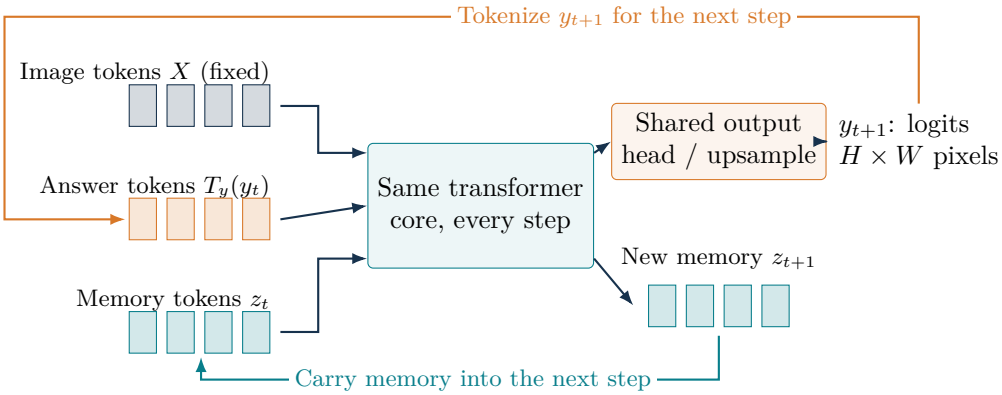


Figure 3: One update, inspired by TRM's input/answer/latent separation [3]. Each small rectangle is a *vector token*, not one pixel or a transformer layer. Token counts and fusion are design choices. This is a segmentation adaptation, not TRM's exact nested schedule.

Mandatory for the main competition

- Reuse one core and all adapters; carry z_t and feed back y_t. All student parameters count toward the budget. A fixed multi-layer core is allowed; independent per-step cores are not.
- Run exactly K updates with one checkpoint. At $K = 0$, return the untouched staff prediction $\sigma(y_0)$. Freeze staff weights and normalization statistics.
- No independent trainable U-Net/CNN segmentation bypass. Convolutions for tokenization, upsampling, and state-conditioned detail recovery are allowed.

4 Design choices and a feasible starting route

Choice	Reasonable starting point	Research directions you may explore
State	Low-resolution feature grid	Two states; multiscale grids; learned memory tokens.
Attention	Local/window attention with image features	Coarse global attention; cross-attention; sparse links. Explain long-range communication.
Output update	Small correction to logits	Replacement logits; gated or uncertainty-aware updates. Residuals are optional.
Feedback	Previous logits and persistent state	Probability/uncertainty embeddings; learned fusion; feedback ablations.
Training	Short unroll, intermediate supervision	Random K; curriculum; truncated gradients; topology/boundary losses.
Stopping	Fixed exact K for ranking	Adaptive halting as a separate experiment, never in ranking mode.

A first implementation, before ambitious changes

1. Verify raw labels and reproduce the frozen baseline on development data. Save both correct and failed road predictions.
2. Start with a 1/16-resolution state, a small shared transformer, and a simple output head. Use crop-based training, not full-resolution global attention.
3. Overfit a tiny training subset to check gradients, ignore masks, and feedback. Then train a first $K = 1, 2, 4$ model on the allowed training partition.
4. Change one meaningful design choice; evaluate the same checkpoint at all K. Track quality, time, memory, and failure cases.

For a starting training objective, let $g(u) \in \{0, 1\}$ be the ground-truth road label, V the valid pixels, and $\mathcal{T}$ the supervised steps. The prediction is $p_t = \sigma(y_t)$:

$$L_{\mathrm{BCE}}(p, g) = -\frac{1}{|V|} \sum_{u \in V} \left[g(u) \log p(u) + (1 - g(u)) \log(1 - p(u)) \right], \tag{5}$$

$$L = \sum_{t \in \mathcal{T}} w_t L_{\mathrm{BCE}}(p_t, g), \qquad w_t \geq 0, \quad \sum_t w_t = 1. \tag{6}$$

Use a numerically stable logits-based implementation; skip all-ignore crops. w_t are training weights chosen by your team, not leaderboard weights. Soft-clDice is an optional differentiable training loss [5]; the hard evaluation skeleton in Section 5 is not differentiable.

Memory matters on one GPU

The target hardware is one NVIDIA L40S (48 GB device memory) [7]. A 512×512 crop at stride 16 has $32^2 = 1{,}024$ tokens. At full pixel resolution it has 262,144 tokens. A dense attention matrix grows as N^2 per head; recurrence does not remove that cost. Training also stores activations across steps.

Proposed design targets, pending a staff pilot: at most 5M core parameters and 10M total student trainable parameters; aim for at most 32 GiB peak allocated training memory. Frozen staff weights are reported separately. Use mixed precision, small batches/gradient accumulation, window attention, or activation checkpointing as needed. Staff must announce final hard caps, GPU-hours, input/tile policy, and timeout before launch. No multi-GPU or outside-model advantage in the main track.

5 Evaluation I: overlap and centerlines

Inputs, labels, and counting conventions

Evaluation uses original-resolution masks, without resizing the ground truth. In the supplied label files, 3 means road, 0 means no-data/ignore, and 1,2,4,5,6,7 are non-road. Use the course-provided files and this encoding; do not apply a library's label remapping twice.

For image i, let $L_i(u)$ be its raw label and define sets

$$V_i = \{u : L_i(u) \neq 0\}, \quad G_i = \{u : L_i(u) = 3\}, \quad P_i = \{u : p_K^{(i)}(u) \geq 0.5\} \cap V_i. \tag{7}$$

G_i is the ground-truth road set; P_i is the predicted road set. The threshold is always 0.5. Ignore pixels never become positive or negative examples. Below, D is a specified collection of images; $|M|$ is the number of pixels in a set M. Counts are summed across images *before* division.

1. Road intersection-over-union: how much road area is correct?

Define $\mathrm{TP} = \sum_i |P_i \cap G_i|$, $\mathrm{FP} = \sum_i |P_i \setminus G_i|$, and $\mathrm{FN} = \sum_i |G_i \setminus P_i|$, where each sum is over $i \in D$. Then

$$\mathrm{IoU}(D, K) = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP} + \mathrm{FN}} = \frac{\sum_{i \in D} |P_i \cap G_i|}{\sum_{i \in D} |P_i \cup G_i|}. \tag{8}$$

For example, 80 correctly predicted road pixels, 10 false road pixels, and 10 missed road pixels give $80/(80 + 10 + 10) = 0.80$. Correctly classified background does not inflate this score.

2. clDice: do the predicted and true centerlines stay inside roads?

Skeletonization thins a binary region to a centerline. Let $S(M)$ be the 2D Zhang skeleton of M, as implemented by the pinned evaluator. Define

$$T_p = \frac{\sum_{i \in D} |S(P_i) \cap G_i|}{\sum_{i \in D} |S(P_i)|}, \qquad T_s = \frac{\sum_{i \in D} |S(G_i) \cap P_i|}{\sum_{i \in D} |S(G_i)|}, \tag{9}$$

$$\mathrm{clDice}(D, K) = \frac{2 T_p T_s}{T_p + T_s}. \tag{10}$$

T_p asks whether the predicted centerline lies in true road; T_s asks whether the true centerline is covered by predicted road. Their harmonic mean penalizes a weak value of either one [5].

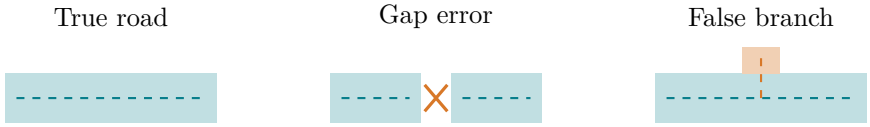


Figure 4: Conceptual masks, not model results. Area overlap can hide a small but consequential gap. Centerline agreement adds a different view, but is not a direct measurement of global routability.

6 Evaluation II: boundaries and exact aggregation

3. Boundary IoU: are road edges in the right place?

Let $E_3(M)$ erode a binary set M using a 3×3 all-ones square for exactly three iterations, treating locations outside the image as zero. Erosion removes the outer layer of a foreground region at each iteration. Define the valid interior $C_i = E_3(V_i)$ and the inner boundary band

$$B_i(M) = (M \setminus E_3(M)) \cap C_i. \tag{11}$$

This gives a three-pixel band in the square-grid distance. C_i excludes a three-pixel neighborhood of no-data and the image frame, so those artificial cuts are not scored as road edges. This is the course's fixed-width Boundary IoU variant, motivated by Cheng et al. [6]; it is not a tunable tolerance.

$$\mathrm{BIoU}(D, K) = \frac{\sum_{i \in D} |B_i(P_i) \cap B_i(G_i)|}{\sum_{i \in D} |B_i(P_i) \cup B_i(G_i)|}. \tag{12}$$

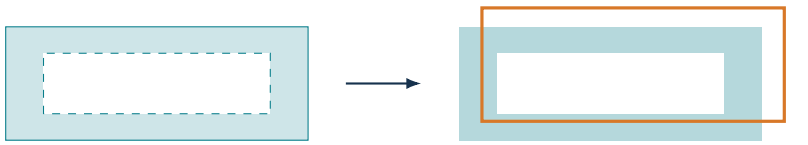


Figure 5: The boundary metric measures overlap of narrow foreground edge bands, not full road interiors. The drawing is schematic; the code uses exactly three erosion iterations.

One quality number for any evaluation set and loop count

All three metrics are fractions in $[0, 1]$, with larger values better:

$$\boxed{A(D, K) = 0.50\,\mathrm{IoU}(D, K) + 0.30\,\mathrm{clDice}(D, K) + 0.20\,\mathrm{BIoU}(D, K).} \tag{13}$$

The weights balance road area, centerline coverage, and boundaries. This is a *course-designed composite*, not a standard multiclass segmentation benchmark score. There are no additional hidden quality or robustness multipliers.

Conventions that prevent implementation disagreements

- Pool integer counts within D first; do not average per-image scores. Empty tiles therefore do not each earn a separate perfect score. Overall scoring pools urban and rural together; also report both domains separately.
- If an IoU union is empty, its value is 1. For T_p or T_s, an empty denominator gives 1. If $T_p + T_s = 0$, clDice is 0. Thus both road sets empty gives clDice 1; only one empty gives 0.
- All-ignore images add zero counts. Reject an entire subset with no valid pixels. Skeletonize each valid-clipped mask separately; never connect neighboring tiles. No-data may cut centerlines, so interpret results near missing data cautiously.
- Use hard Zhang skeletons, not a soft training approximation. A high clDice does not establish graph connectivity, road navigability, or human-like reasoning. Qualitative and controlled experiments remain necessary.

7 The public leaderboard formula

Let D_{ordinary} contain unmodified evaluation images. Let D_{stress} contain one controlled corrupted version of each of those images, with its original label. Section 8 defines the corruptions. The same four quality measurements determine every team's score:

$$\boxed{\begin{aligned}\text{Score} = 100\big[&0.20A(D_{\text{ordinary}}, 1) + 0.20A(D_{\text{ordinary}}, 2) \\ &+ 0.40A(D_{\text{ordinary}}, 4) + 0.20A(D_{\text{stress}}, 4)\big].\end{aligned}} \tag{14}$$

The range is 0–100. The first two terms reward useful early predictions; the third emphasizes the four-step result; the last tests robustness. **All weights, definitions, thresholds, and corruption ranges are public.** Only private evaluation instances and seeds are withheld.



Figure 6: How the final score allocates its weight. Each segment uses the same quality definition in Eq. (13).

A complete numerical example

Suppose ordinary $K = 4$ gives road IoU 0.72, clDice 0.80, and Boundary IoU 0.40. Then

$$A(D_{\text{ordinary}}, 4) = 0.50(0.72) + 0.30(0.80) + 0.20(0.40) = 0.68.$$

If the other qualities are $A(D_{\text{ordinary}}, 1) = 0.60$, $A(D_{\text{ordinary}}, 2) = 0.64$, and $A(D_{\text{stress}}, 4) = 0.55$, then

$$\text{Score} = 100[0.20(0.60) + 0.20(0.64) + 0.40(0.68) + 0.20(0.55)] = 63.0.$$

These are illustrative values, not measured baseline performance.

What is reported but does not add another score multiplier?

- $K = 0$ staff baseline; $K = 6, 8$ extrapolation; every component at every K.
- Refinement gain $\Delta_K = A(D_{\text{ordinary}}, K) - A(D_{\text{ordinary}}, 0)$ and $A(D_{\text{ordinary}}, 4) - A(D_{\text{ordinary}}, 1)$. Gains may be negative. There is no reward for deliberately weakening an early prediction.
- Urban/rural breakdown; ordinary/stress breakdown; parameter counts; latency; inference and training peak memory; training GPU-hours.

Eligibility and quality are separate. A strong U-Net without the required shared loop is not an eligible main entry. An eligible loop that fails to improve can still support a strong research report. No monotonic-improvement threshold is an eligibility test.

8 Stress tests, data splits, and competition rules

A reproducible, disclosed stress suite

Staff generate exactly one stress version per source tile. Within each domain, a seed-based permutation assigns shadow, occlusion, and blur as evenly as possible (counts differ by at most one). Locations do not use labels. The published `make_stress.py` is the reference implementation:

Family	Transformation at original image resolution
Shadow	One rectangle, requested area fraction uniformly in $[0.04, 0.12]$ and aspect ratio in $[0.5, 2]$. Multiply RGB inside by a factor uniformly in $[0.3, 0.6]$.
Occlusion	One rectangle, area fraction uniformly in $[0.005, 0.02]$, aspect ratio in $[0.5, 2]$. Fill with the source image's per-channel median.
Blur	Gaussian blur of each RGB channel; standard deviation uniformly in $[1, 2]$ pixels, reflected padding, kernel truncated at four standard deviations.

For rectangle area fraction a and width/height ratio r, choose height $h = \mathrm{round}(\sqrt{aHW/r})$ and width $w = \mathrm{round}(\sqrt{aHWr})$, each clamped to the image and at least one pixel. Sample its top-left corner uniformly over fitting integer locations. Clip RGB to $[0, 1]$, multiply by 255, and round to save an 8-bit image. Seeds and exact rounding are fixed by the code and pinned dependencies.

The ground truth is unchanged; the frozen segmenter runs on the *corrupted RGB input*. There is no oracle clean mask, clean feature cache, or paired clean image available during a stress run. Publish development examples and seeds. The stress set is a synthetic robustness test, not a substitute for natural domain shift. Some occluded content is genuinely unknowable; do not infer that every gap should be connected.

The course-provided data

Staff will provide a versioned collection of high-resolution RGB satellite-image tiles and pixel-aligned land-cover masks. Tiles are 1024×1024 pixels at approximately 0.3 m per pixel. Urban scenes contain dense buildings, intersections, and roads of different widths; rural scenes include fields, vegetation, water, and smaller roads. Shadows, visually similar surfaces, and different scene layouts make road extraction challenging.

The task is **binary road versus non-road segmentation**: other land-cover classes become non-road, and no-data is ignored. Download the course package from staff's announced location. Use its image IDs, checksums, and fixed split manifests. Keep every crop from a tile in that tile's assigned split. Do not obtain substitute images or annotations elsewhere.

Allowed use. Fit model weights on training data and choose settings/checkpoints on development data. Evaluation annotations are reserved for staff. Do not search for matching images or annotations elsewhere, reconstruct evaluation labels from external sources, or use evaluation labels for training or tuning. No extra labeled data, outside segmentation checkpoints, or test-time label access is allowed. Full data provenance and research-citation instructions will be supplied by staff for any publication; the course package does not imply staff created the underlying imagery.

Public versus final ranking

Proposed policy: reserve 30% of course evaluation tiles for public feedback and 70% for private ranking, stratified by domain. Compute Eq. (14) separately on each partition; do not blend their scores. Allow two submissions per team per day. Nominate *one* checkpoint before private evaluation. Rank by full-precision private score, not displayed rounding. Staff rerun entries offline with resource limits.

9 Submission interface: staff choose the loop count

One command, one checkpoint, exactly K updates

The staff-owned runner handles image loading, repetition, output validation, and saving. Your team implements the model adapter in `submission/model.py` and submits safe tensor weights. Example official command, once the staff baseline is released:

```
python predict.py --checkpoint submission/model.safetensors \
  --baseline staff/baseline.safetensors \
  --input evaluation/images --output predictions/k4 \
  --steps 4 --device cuda --save-trace
```

`-steps 0` returns the staff baseline, not your initializer's prediction. Ranking uses 1, 2, and 4. The runner also accepts 6 and 8, and never sends the requested total K to your model. An adaptive-stopping experiment must use a separate interface; it cannot alter this command's exact-step meaning.

Student interface	Responsibility
`build_refiner(...)`	Construct all trainable modules once. Expose the shared transformer as `model.core`. Load the same state dictionary for all K.
`initialize(f, y0)`	Return the initial hidden state from features and initial logits. Do not call the recurrent core or run a separate full segmenter here.
`step(f, z, y)`	Take features, latent tokens z_t, and answer logits y_t; call the same core once; return z_{t+1} and full-resolution y_{t+1}. No labels or total K.

Features are staff-defined tensors; the demo uses five channels on a coarse grid. Logits must have shape $B \times 1 \times H \times W$. This starter runner processes one complete image at a time; any approved internal tiling must be deterministic, declared, and counted in compute. Staff must freeze the feature/tile contract before teams train.

Files produced and checked

For input `images/urban/001.png`, the ID is `urban/001`. Output includes a float32 $H \times W$ probability array at `probabilities/urban/001.npy`, a 0/255 PNG mask, and `run.json`. Probabilities must be finite and in $[0, 1]$. With `-save-trace`, predictions $0, \ldots, K$ are saved. Evaluation thresholds probabilities at $p \geq 0.5$, ignoring the submitted PNG.

The run manifest records image/checkpoint hashes, source hash, actual core calls, parameter counts, and diagnostic timing/memory. Missing, duplicate, mismatched, or invalid outputs fail validation. Hooks check core calls and parameter identity; they are not a proof against dishonest code. Staff also inspect source and run controlled interventions. No network or evaluation-label access is permitted.

Try the included smoke test before real training

```
python smoke_test.py --output smoke_outputs
python -m unittest discover -s tests -v
```

The smoke test creates random weights and synthetic images, runs all four score components, and checks the combined result. It does not train a model. Official mode requires a staff baseline adapter. See the README for individual commands and manifests. Do not edit staff inference/evaluation code for official submissions.

10 Experiments: demonstrate understanding, not just accuracy

Required comparisons

1. **Baseline and iteration curve.** Show the frozen baseline and your *same* checkpoint at $K = 1, 2, 4, 6, 8$. Plot each quality component against both K and measured time. State which step counts were used in training.
2. **State/feedback interventions.** At inference, reset the hidden state before every update; separately replace mask feedback with the initial logits. Compare to the unmodified model. These interventions can create distribution shift, so treat them as diagnostics, not conclusive causal proof. Include a retrained no-feedback control when making a strong claim about learning to use feedback.
3. **A non-recurrent control.** Train a one-step model with approximately matched student parameter count. Also compare with an untied stack at approximately matched computation if feasible. A single control will generally not match both compute and parameters; report the mismatch rather than claiming it does.
4. **One hypothesis-driven design ablation.** Examples: local versus global context, residual versus replacement logits, intermediate losses, or a loop-count curriculum. Change one major factor and explain the expected outcome before running it.
5. **Failures and uncertainty.** Show unselected successes and failures, including false connections, disappearing roads, and later-step degradation. Report urban/rural and stress-family behavior. Where affordable, use three training seeds for the main claim; otherwise disclose the limitation and use paired image bootstrap intervals, noting spatial correlation.

How staff will check that it is genuinely a loop

- Trace the computation at two different K: identical parameter objects/storage, repeated calls to one core, a changing carried state, and explicit mask feedback.
- Check prefix consistency: steps 0–2 from a $K = 4$ run should match a separate $K = 2$ run with the same image/checkpoint/seed. The model must not choose a special branch for the final requested step.
- Ask each team member to explain one update, distinguish state from parameters, follow tensor shapes, and predict what a state-reset or feedback removal should change.

Merely calling a transformer and discarding its output does not satisfy the project. Neither does allocating a new independent transformer per iteration. A module list is not automatically wrong: actual parameter reuse and dataflow, not the class name, decide compliance.

What makes a strong research contribution?

A focused, falsifiable claim supported by fair comparisons: for example, *"Shared refinement helps recover thin road segments under shadows at the same inference budget, but over-connects parallel roads when unrolled too far."* Explain the mechanism, alternative explanations, and limitations. A leaderboard improvement alone does not establish novelty or reasoning.

Promising teams can extend the study with another public road dataset, stronger compute-matched controls, multiple seeds, and an external holdout. Those are possible paper extensions, not a requirement to complete the course project. A submission-quality result is an aspiration; a well-executed negative result can earn an excellent course grade.

11 Expectations, milestones, and grading

Teams and milestones

Project A teams have **three students**. Share implementation, experiments, and interpretation; rotate roles so every member can explain the whole pipeline. Keep a contribution log and identify a primary owner and reviewer for each major experiment.

When	Expected work
September 28	Project Pitch Day (syllabus). Form a focused question and identify the first minimal experiment.
Before November 2	Suggested ungraded checkpoint: label/CLI sanity tests, baseline reproduction, one shared update, and memory measurement. Staff should offer a short early transformer/recurrence tutorial.
November 2	Proposal and midway update (syllabus): question, method diagram, initial results, resource plan, ablations, risks, and each member's contribution.
December 14–18	Final presentation, report, code, and weights; exact date/time to be confirmed. No project assessment is due in the December 11–13 study period.

The syllabus places RNNs on November 16 and Transformers I on November 18, after the midway deadline. Do not wait for those lectures to begin: use this primer, starter interface, and the proposed early tutorial.

Proposed project rubric (100 points, subject to instructor approval)

Points	Component	What earns credit
10	Question and proposal	Specific hypothesis, relevant reading, clear design and feasible plan.
15	Midway work	Working loop, verified data pipeline, honest initial results, sensible response to failures.
30	Final research report	Correct method, fair controls, meaningful ablations, uncertainty, error analysis, limitations, and supported conclusions.
15	Reproducible implementation	Runnable code/weights, exact-K behavior, environment, logs, compute accounting, and clean evaluation separation.
15	Presentation and individual understanding	Clear explanation and live questions; every member understands recurrence, metrics, and their contributions.
15	Leaderboard component	Valid final entry plus competitive ranking, defined below.

For a valid final entry, assign $10 + 5(N - r)/(N - 1)$ leaderboard points, where $N > 1$ is the number of eligible teams and r is rank (1 is best). Use mean ranks for exact ties. If $N = 1$, assign 15. An invalid/missing entry receives 0 in this component, not automatically 0 for the entire project. Thus the best and worst eligible teams receive different ranking credit, while rigorous science remains most of the grade. The syllabus assigns the project 30% of the course grade.

Academic integrity and use of AI

Follow the syllabus: AI may assist coding and research, but students own the creative decisions and must understand every submitted line. Write the report yourselves; AI use is limited to grammatical clarity. AI is not permitted for in-class presentations or quizzes. Cite external code, datasets, figures, and ideas; disclose tools and assistance. Do not share held-out labels, inspect evaluation annotations, or disguise pretrained/external models. Course late and academic-integrity policies apply.

12 Reading guide and final checklist

Three papers to understand the central idea

1. **Universal Transformers** [2]: read the shared-depth update and its diagram. Ask: what changes between iterations, and what is reused?
2. **Less is More: Recursive Reasoning with Tiny Networks** [3]: focus on latent-state and answer refinement. Its reasoning results do not establish that the approach works for road segmentation; that transfer is your research question.
3. **Recurrent Scene Parsing with Perspective Understanding in the Loop** [4]: an earlier example of recurrent visual inference. Compare its feedback mechanism with your design. It is not itself a looped transformer.

Before the final submission

- One nominated checkpoint, source code, pinned environment, training instructions, and a run manifest linking results to seeds/checkpoints.
- A method diagram and parameter table; K-sweep and latency/memory measurements; all required controls; representative intermediate masks and failures.
- Original report and slides in the course templates, with citations, limitations, contribution statement, and AI/code-use disclosures. Follow the course page limits when announced; do not replace evidence with architecture complexity.
- Confirm that a fresh offline environment can run the exact command, with the same weights at all K, no downloads, and no labels available to inference.

References

[1] J. Wang, Z. Zheng, A. Ma, X. Lu, and Y. Zhong. Source publication for Figure 1. NeurIPS Datasets and Benchmarks, 2021. Original publication and figure attribution; CC BY-NC-SA 4.0. Display crops only; no claim of original imagery authorship by course staff.

[2] M. Dehghani, S. Gouws, O. Vinyals, J. Uszkoreit, and L. Kaiser. *Universal Transformers*. ICLR, 2019. https://arxiv.org/abs/1807.03819.

[3] A. Jolicoeur-Martineau. *Less is More: Recursive Reasoning with Tiny Networks*. arXiv, 2025. https://arxiv.org/abs/2510.04871.

[4] S. Kong and C. C. Fowlkes. *Recurrent Scene Parsing with Perspective Understanding in the Loop*. CVPR, 2018. https://arxiv.org/abs/1705.07238.

[5] S. Shit et al. *clDice – A Novel Topology-Preserving Loss Function for Tubular Structure Segmentation*. CVPR, 2021. https://arxiv.org/abs/2003.07311.

[6] B. Cheng, R. Girshick, P. Dollár, A. C. Berg, and A. Kirillov. *Boundary IoU: Improving Object-Centric Image Segmentation Evaluation*. CVPR, 2021. https://arxiv.org/abs/2103.16562.

[7] NVIDIA. *L40S GPU specifications*. https://www.nvidia.com/en-us/data-center/l40s/. Accessed August 29, 2026.